LECTURE 12

# Adapting Pre-Trained Models to New Tasks

In the last lecture, we saw how to use sampling techniques to generate programs from a language model, subject to some contraints. In general, such techniques can be very useful when the program we are looking for is not too unlikely to be generated by the model. However, if the program is very unlikely, or even has zero probability, then we will need to go beyond sampling techniques and instead adapt the model itself to the task at hand. In this lecture, we will look at a few different ways to adapt a pre-trained model to a specific task, using prompts and weights. These techniques are most effective when we want to adapt a model to more specialized tasks, such as writing programs in a new programming language or domain-specific language, or simply to a more specialized domain.

## 12.1 Instructing Large Language Models

One of the most surprising features of large language models is their ability to incorporate new information provided to them through their prompt. One way this comes into play in the context of program synthesis is instruction following. Large language models are explicitly trained to follow instructions provided in the prompt, and this can be used to guide the model towards generating programs that meet certain criteria, and even to provide it with step-by-step instructions on how to solve a problem.

An interesting example of this approach is Grammar Prompting [1]. At a high-level, the idea is to help the language model to generate programs in a domain specific language (DSL) by providing it with a grammar for the DSL in the prompt, but the details are a bit more subtle. What they found was that simply providing the grammar in the prompt was not enough to get the model to generate valid programs. In addition to providing the grammar, the model is instructed to produce a *specialized grammar* for the solution before producing the solution itself. A specialized grammar is simply a subset of the original grammar that only contains the rules that are needed to produce the solution. One way to view this is that the LLM is playing the role of the recognition model from Lecture 9 by first identifying the relevant components from the grammar, and then using these to produce the solution. An added benefit of this approach is that the specialized grammar can be used to do constrained

decoding, thus further improving the chances of generating a valid solution.

# Grammar Prompting

Bailin Wang, Zi Wang, Xuezhi Wang, Yuan Cao, Rif A Saurous, Yoon Kim, Grammar Prompting for Domain-Specific Language Generation with Large Language Models, NeurIPS 2023

**LLM Prompt**

You are an expert programmer, and you need to write a program for the given natural language query. First, you should write a grammar that contains all the necessary BNF rules. Then, you should write programs that conform to your predicted rules.

(optional) $G$: [BEGIN RULES] ... [END RULES]

$x^{(1)}$: find the meeting on Wednesday with Bob and Carol

$G[y^{(1)}]$:

```
event      ::= "QueryEvent(" constraint ")"
constraint ::= "(&" constraint constraint ")"
             | "(start_?" date ")"
             | "(attendee_?" attendee attendee ")"
date       ::= "Wednesday"
attendee   ::= "Bob" | "Carol"
```

$y^{(1)}$: `QueryEvent(& (start_? Wednesday)(attendee_? Bob Carol))`

...

$x$: Add meeting with Jean's manager on Wednesday at 3PM

**LLM Output**

$\hat{G}$:

```
event      ::= "CreateEvent(" constraint ")"
constraint ::= "(&" constraint constraint ")"
             | "(start_?" date time ")"
             | "(attendee_?" attendee ")"
date       ::= "Wednesday"
time       ::= "NumberPM(3)"
attendee   ::= "FindManager(" attendee ")" | "Jean"
```

$\hat{y}$: `CreateEvent(& (start_? Wednesday NumberPM(3))(attendee_? FindManager(Jean)))`

The full set of instructions is shown in the figure (copied from the paper). The first part of the prompt provides the grammar, and then the second part instructs the model to first produce a specialized grammar, and then to produce a program that is valid according to this specialized grammar. Multiple examples of this process are provided in the prompt, and then the model is asked to produce a solution for a new problem.

## 12.2 Retreival-Augmented Generation

A fixed set of examples can be enough for the model to learn what it needs to know to solve a particular problem, but in general, the closer the given examples are to the original problem, the more useful those examples will be in guiding the model to the desired solution. *Retrieval Augmented Generation* (RAG) exploits this insight by retrieving relevant examples from a large database of examples, and then providing these to the model in the prompt. The idea was first introduced in 2020 for general NLP tasks requiring knowledge not available at training time [2]. The idea was applied to code generation shortly after in 2021 [3].

Formally, RAG assumes that we have a pre-trained language model $p_\theta$ and a retrieval model $r_\phi$. We then begin by processing our dataset, which consists of a set of documents $D = \{d_1, d_2, \ldots, d_n\}$, into a set of embeddings $E = \{e_i \triangleq r_\phi(d_i) \mid i = 1, \ldots, n\}$. These can then be stored in a key-value store, where the keys are the embeddings $e_i$ and the values are the documents $d_i$. When we want to generate a response to a query $q$, we then embed the query using

the retrieval model, $e_q \triangleq r_\phi(q)$, and then retrieve the top $k$ documents that are most similar to the query embedding. The notion of similarity can be defined in many ways, but the most common is to use cosine similarity:

$$\text{similarity}(e_q, e_i) = \frac{\langle e_q, e_i \rangle}{\|e_q\| \|e_i\|}.$$

\] The top $k$ documents are then concatenated into a prompt which is passed to the pre-trained language model in order to generate a response.

Thus, RAG is very similar to in-context learning, but with two key differences:

- Historically, the context windows of transformers have been limited to a few thousand tokens, making it difficult to put all relevant information in the prompt. This has been mitigated to some extent by the development of long-context transformers, but these remain limited to perhaps 64k or 128k tokens. In RAG, the amount of relevant information that can be made use of is much larger, as it is not limited by the context window of the model; instead, the only bottleneck is the speed and the accuracy with which the retrieval system can return the relevant documents.
- With RAG, the retrieval model also serves as a filter. This can reduce the amount of irrelevant information that is passed to the model, which can have a significant impact on performance.

While it may appear to be somewhat brutish of a solution, RAG has proven to be very effective in practice. In addition to its simplicity, RAG benefits from being trivial to extend to more data in an online fashion, as we can simply add more documents to the retrieval database without needing to retrain the model.

## 12.3 Finetuning

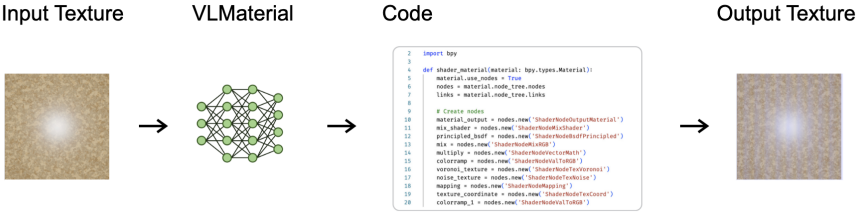
The most straightforward way to adapt a pre-trained model to a specific task is to finetune it on a dataset that is relevant to the task. Finetuning really just means that we train the model on a new dataset, but starting from the pre-trained weights instead of from scratch. Oftentimes, this lets us get away with training on relatively little data, since the base model already has a lot of general capabilities.

Unlike the sampling techniques we saw in the last lecture, finetuning fundamentally changes the model itself. This means that even if the desired program had zero probability under the original model $p_\theta$, it may have high likelihood under the finetuned model $p_{\theta'}$.

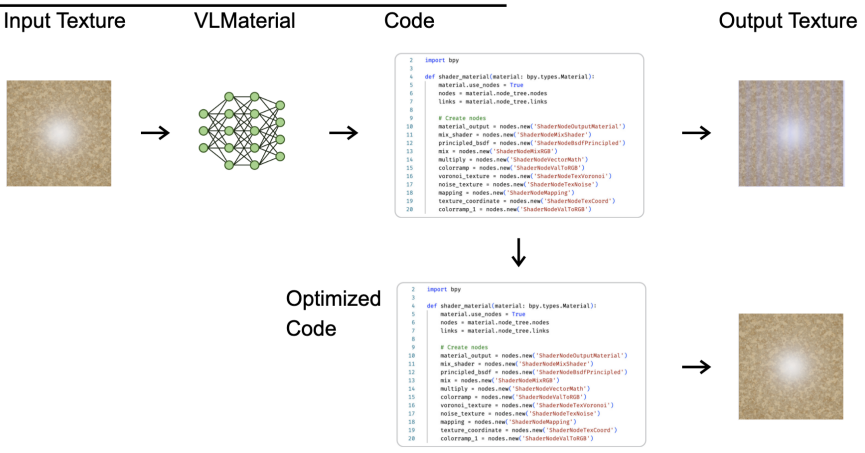
## Procedural Materials

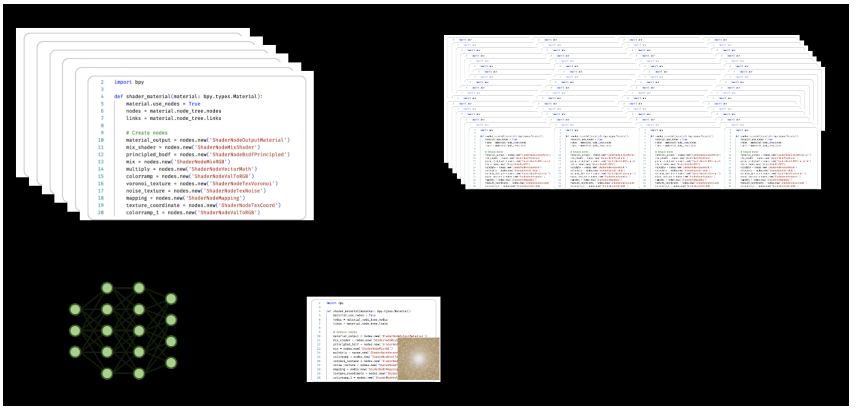



## VLMaterial: Basics

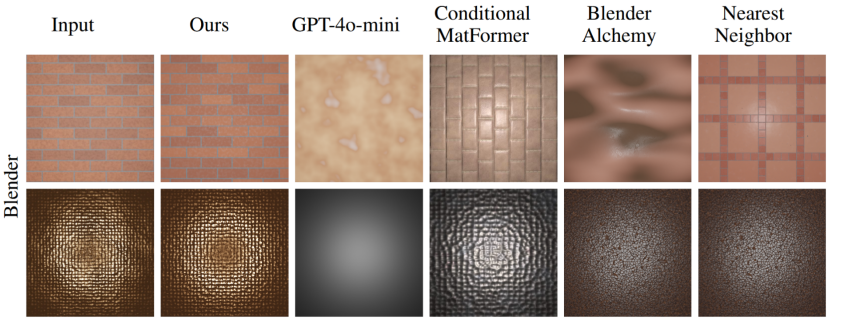



## VLMaterial: Basics

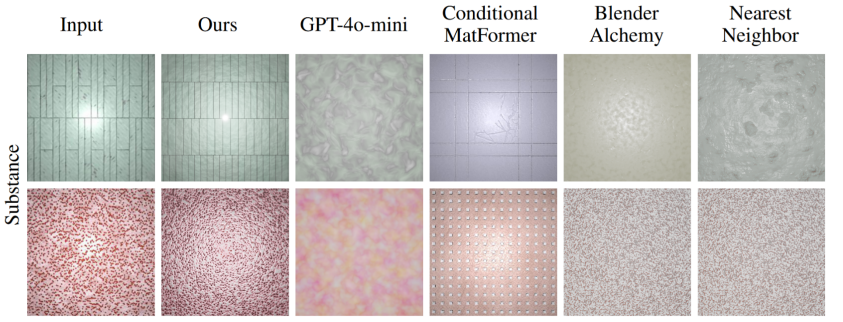


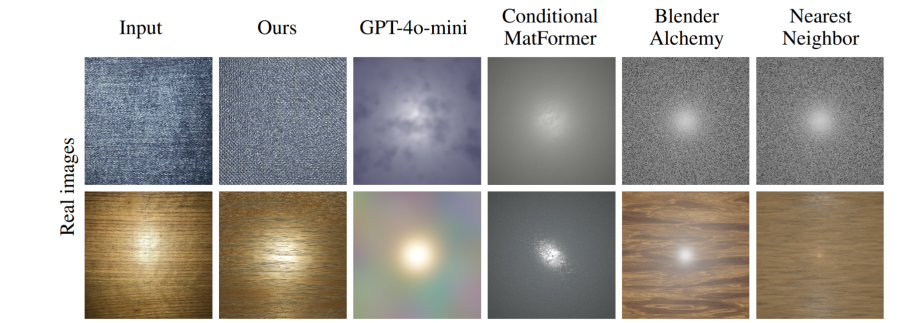


# Experiment Results




# Experiment Results






As an example, a couple of years ago, my colleague Wojciech Matusik and I got interested in the use of language models to synthesize texture programs that match an image of a given texture [4]. Even though textures in modern graphics systems are often designed through drag-and-drop interfaces, the Blender system has a python API that allows you to develop textures programmatically, so in principle, this would be a good target for a vision language model. However, there is just not enough training data in the wild for an off-the-shelf model to be any good at this task, but by finetuning a model on a synthetically generated dataset of texture programs, we were able to get it to generate textures from images very effectively.

While finetuning is a powerful technique, it does have some drawbacks. First, it requires access to a dataset that is relevant to the task at hand, which may be difficult to construct in cases such as the above. Secondly, as a form of continual learning, finetuning can lead to catastrophic forgetting, where the model forgets how to perform tasks it was previously able to do. In the worst case, this can mean that the model loses some of its general capabilities that are actually useful in the domain, such as understanding the natural language feedback from the user. Finally, finetuning can be computationally expensive, as it requires training the entire model. While the first two of these issues can be mitigated to some extent by careful dataset construction and training, the last one is a fundamental limitation of finetuning. As a result, several parameter-efficient variations have been developed. We are going to explore two in particular: Prompt tuning and low-rank adaptation (LoRA).

## 12.4 Why parameter efficient?

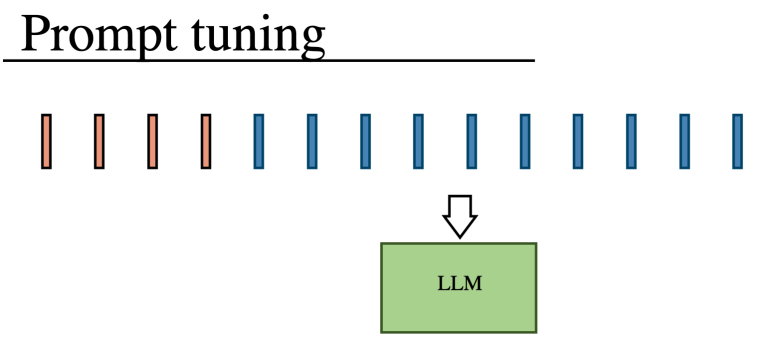
Before we dive into these techniques, let's briefly discuss why parameter-efficient finetuning is important. Suppose that you have a model that takes $M_{model}$ bytes of storage to execute; optimizing the entire model requires at

least $4 \times M_{model}$ bytes of memory. This is because in addition to the original weights, we need to store gradients for each parameter, and then Adam optimizer, the most commonly used optimization algorithm) needs to store two additional copies of the weights.

In contrast, suppose that only $M_{adapt}$ bytes of parameters need to be optimized. Then, the total memory requirement is only $M_{model} + 3 \times M_{adapt}$ bytes. This means that if $M_{adapt} \ll M_{model}$, we can significantly reduce the memory requirements for finetuning. This is particularly important when working with very large models.

Parameter-efficient finetuning also has other benefits. Since fewer parameters are being optimized, it is less likely that the model will forget how to perform tasks, and if you want to deploy multiple adapted models, each specialized to a different task, the storage requirements are significantly reduced as well.

## 12.5 Prompt tuning




Keep the model the same, create new tokens that you tune to produce the desired output

The first technique we will consider is prompt tuning [5]. The idea behind prompt tuning is to learn a small set of continuous vectors that are prepended to the input prompt, and which are optimized to guide the model towards generating the desired output. This can almost be seen as a form of in-context learning, but where the context is learned instead of being provided by examples. A big advantage is that it does not require any changes to the model itself, significantly reducing the risk of catastrophic forgetting.

A closely related technique is prefix tuning [6], where instead of prepending learned vectors to the input, we prepend them to the hidden states of each layer in the model. This allows for more fine-grained control over the model's behavior, but requires modifying the model itself, which can be more complex to implement.

There have been other embelishments to the basic prompt tuning idea. For example, in a 2024 paper, Jain et al. propose to combine the fixed prefix with a per-task prefix that is generated by a small model [7]. The paper provides a detailed empirical evaluation comparing the approach with traditional prompt tuning, full fine tuning and LoRA (which we discuss shortly).

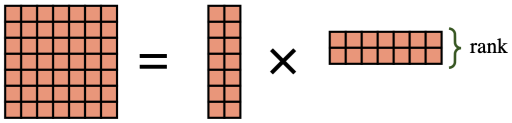
### 12.6 Low-Rank Adaptation (LoRA)

## LoRA: basics

$$Attn(q, K, V) = W_v \cdot V \cdot softmax(W_K \cdot K^T \cdot W_q \cdot q)$$

$$W_{new} = W_{old} + \Delta W$$

$$\Delta W = B \cdot A$$




## Which weights to adapt?

| | # of Trainable Parameters = 18M | | | | | | |
|---|---|---|---|---|---|---|---|
| Weight Type | $W_q$ | $W_k$ | $W_v$ | $W_o$ | $W_q, W_k$ | $W_q, W_v$ | $W_q, W_k, W_v, W_o$ |
| Rank $r$ | 8 | 8 | 8 | 8 | 4 | 4 | 2 |
| WikiSQL (±0.5%) | 70.4 | 70.0 | 73.0 | 73.2 | 71.4 | **73.7** | **73.7** |
| MultiNLI (±0.1%) | 91.0 | 90.8 | 91.0 | 91.3 | 91.3 | 91.3 | **91.7** |

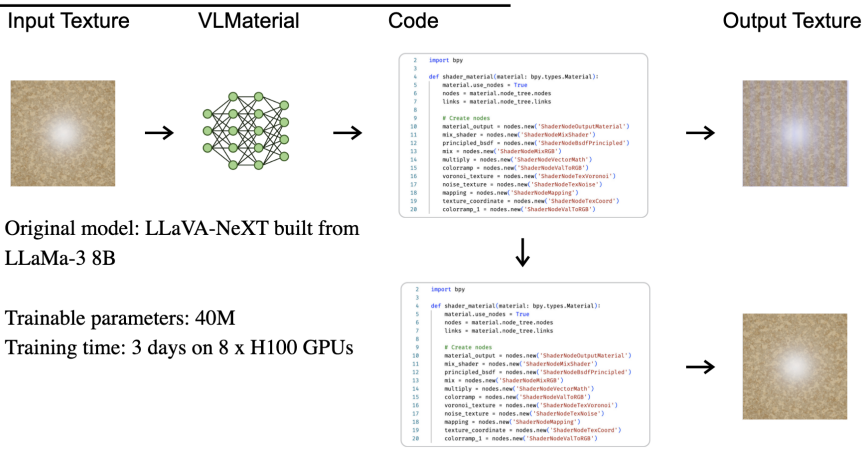
From the LoRA paper (ICLR 2022)

# Chosing rank

| | Weight Type | $r = 1$ | $r = 2$ | $r = 4$ | $r = 8$ | $r = 64$ |
|---|---|---|---|---|---|---|
| WikiSQL($\pm 0.5\%$) | $W_q$ | 68.8 | 69.6 | 70.5 | 70.4 | 70.0 |
| | $W_q, W_v$ | 73.4 | 73.3 | 73.7 | 73.8 | 73.5 |
| | $W_q, W_k, W_v, W_o$ | 74.1 | 73.7 | 74.0 | 74.0 | 73.9 |
| MultiNLI ($\pm 0.1\%$) | $W_q$ | 90.7 | 90.9 | 91.1 | 90.7 | 90.7 |
| | $W_q, W_v$ | 91.3 | 91.4 | 91.3 | 91.6 | 91.4 |
| | $W_q, W_k, W_v, W_o$ | 91.2 | 91.7 | 91.7 | 91.5 | 91.4 |

From the LoRA paper (ICLR 2022)

# VLMaterial: Basics




In LoRA, instead of finetuning the entire model, we keep the model's weights fixed and instead learn a small set of *adapters*. These adapters consist of two low-rank (tall-skinny) matrices which are multipled together and then added to the model's activations.

Consider for example a (single-head) self-attention layer, with weights $W_q, W_k, W_v$ for the query, key, and value projections respectively. Each of these weights is a matrix of size $d \times d$, where $d$ is the layer's hidden dimension; thus, if we were to finetune the model, we would need to update $3d^2$ parameters. In LoRA, we instead learn six low-rank matrices $A_q, B_q, A_k, B_k, A_v, B_v \in \mathbb{R}^{d \times r}$, where $r$ is a small rank (typically much smaller than $d$). We then replace the

original query, key, and value projections with the following:

$$
\begin{aligned}
Q &= x(W_q + A_q B_q^T) = xW_q + xA_q B_q^T \\
K &= x(W_k + A_k B_k^T) = xW_k + xA_k B_k^T \\
V &= x(W_v + A_v B_v^T) = xW_v + xA_v B_v^T
\end{aligned}
$$

where $x$ is the input to the self-attention layer. This means that we only need to update $6dr$ parameters, which is smaller than $3d^2$ as long as $r < d/2$.

In practice, LoRA is often applied to the fully-connected layers of the model as well, which typically have much larger hidden dimensions than the self-attention layers; this is where the largest savings in parameters come from.

LoRA is a very powerful technique, and has come to dominate the field of parameter-efficient finetuning. Since the number of parameters that are updated is much smaller than in standard finetuning, LoRA is faster, requires less data, and is less prone to catastrophic forgetting than standard finetuning. However, intuitively, LoRA can only adapt the model to tasks that are similar to the tasks it was pre-trained on.

# References


[1] Bailin Wang, Zi Wang, Xuezhi Wang, Yuan Cao, Rif A. Saurous, and Yoon Kim. Grammar prompting for domain-specific language generation with large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/cd40d0d65bfebb894ccc9ea822b47fa8-Paper-Conference.pdf.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020. URL https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf. also arXiv preprint arXiv:2005.11401.

[3] Md Rizwan Parvez, Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. Retrieval augmented code generation and summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 2719–2734. Association for Computational Linguistics, November 2021. doi: 10.18653/v1/2021.findings-emnlp.232. URL https://aclanthology.org/2021.findings-emnlp.232/.

[4] Beichen Li, Rundi Wu, Armando Solar-Lezama, Changxi Zheng, Liang Shi, Bernd Bickel, and Wojciech Matusik. Vlmaterial: Procedural material generation with large vision-language models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=wHebuIb6IH.

[5] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3045–3059, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.243. URL https://aclanthology.org/2021.emnlp-main.243.

[6] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4582–4597, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.353. URL https://aclanthology.org/2021.acl-long.353.

[7] Abhinav Jain, Swarat Chaudhuri, Thomas Reps, and Chris Jermaine. Prompt tuning strikes back: Customizing foundation models with low-rank prompt adaptation. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 47297–47316. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/548551c07a68c8f0a87d67c6167cedb1-Paper-Conference.pdf.